Workgroup: fapi
Published: 22 February 2025
Status: Final
Authors: D. Fett, *Authlete*; D. Tonge, *Moneyhub Financial Technology*; J. Heenan, *Authlete*

FAPI 2.0 Security Profile

Abstract

OIDF FAPI 2.0 is an API security profile suitable for high-security applications based on the OAuth 2.0 Authorization Framework [RFC6749].

Foreword

The OpenID Foundation (OIDF) promotes, protects and nurtures the OpenID community and technologies. As a non-profit international standardizing body, it is comprised by over 160 participating entities (workgroup participant). The work of preparing implementer drafts and final international standards is carried out through OIDF workgroups in accordance with the OpenID Process. Participants interested in a subject for which a workgroup has been established have the right to be represented in that workgroup. International organizations, governmental and non-governmental, in liaison with OIDF, also take part in the work. OIDF collaborates closely with other standardizing bodies in the related fields.

Final drafts adopted by the Workgroup through consensus are circulated publicly for the public review for 60 days and for the OIDF members for voting. Publication as an OIDF Standard requires approval by at least 50% of the members casting a vote. There is a possibility that some of the elements of this document may be subject to patent rights. OIDF shall not be held responsible for identifying any or all such patent rights.

Introduction

The FAPI 2.0 Security Profile is an API security profile based on the OAuth 2.0 Authorization Framework [RFC6749] and related specifications that aims to reach the security goals laid out in the Attacker Model [attackermodel] so that it is suitable for protecting APIs in high-value scenarios. It also follows the recommendations in the OAuth Security BCP [RFC9700].

This document specifies the process for a client to obtain sender-constrained tokens from an authorization server and use them securely with resource servers.

The OpenID Foundation FAPI Working Group publishes additional documents that build on this profile as part of the FAPI 2.0 framework.

The security property is formally analysed [FAPI2SEC] under the aforementioned attacker model. For the security assumptions, please refer the attacker model.

While the security profile was initially developed with a focus on financial applications, it is designed to be universally applicable for protecting APIs exposing high-value and sensitive (personal and other) data, for example, in e-health and e-government applications.

Notational conventions

The keywords "shall", "shall not", "should", "should not", "may", and "can" in this document are to be interpreted as described in ISO Directive Part 2 [ISODIR2]. These keywords are not used as dictionary terms such that any occurrence of them shall be interpreted as keywords and are not to be interpreted with their natural language meanings.

Table of Contents

1. Scope

This document provides a general-purpose high security profile of OAuth 2.0 that has been proved by formal analysis to meet the stated attacker model. This document specifies the requirements for:

- Confidential clients to securely obtain OAuth tokens from authorization servers;
- Confidential clients to securely use those tokens to access protected resources at resource servers;
- Authorization servers to securely issue OAuth tokens to confidential clients;
- Resource servers to securely accept and verify OAuth tokens from confidential clients.

2. Normative references

The following documents are referred to in the text in such a way that some or all of their content constitutes requirements of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

See Section 10 for normative references.

3. Terms and definitions

For the purpose of this document, the terms defined in [RFC6749], [RFC6750], [RFC7636], [OIDC] and [ISO29100] apply.

4. Symbols and Abbreviated terms

API – Application Programming Interface

BCM – Basin, Cremers, Meier

BCP – Best Current Practice

CAA – Certificate Authority Authorization

CIBA – Client Initiated Backchannel Authentication

CSRF – Cross-Site Request Forgery

DNS – Domain Name System

DNSSEC – Domain Name System Security Extensions

HTTP – Hyper Text Transfer Protocol

JAR – JWT-Secured Authorization Request

JARM – JWT Secured Authorization Response Mode

JWK – JSON Web Key

JWKS – JSON Web Key Sets

JWT – JSON Web Token

JOSE – Javascript Object Signing and Encryption

JSON – JavaScript Object Notation

MTLS – Mutual Transport Layer Security

OIDF – OpenID Foundation

PAR – Pushed Authorization Requests

PKCE – Proof Key for Code Exchange

QR – Quick Response

RSA – Rivest-Shamir-Adleman

REST – Representational State Transfer

TLS – Transport Layer Security

URI – Uniform Resource Identifier

URL – Uniform Resource Locator

5. Security profile

5.1. Overview

5.1.1. Introduction

The FAPI 2.0 Security Profile is an API security profile based on the OAuth 2.0 Authorization Framework [RFC6749], that aims: - to reach the security goals laid out in the Attacker Model [attackermodel]; and - to follow the recommendations in the OAuth Security BCP [RFC9700].

The OpenID FAPI Working Group is not currently aware of any mechanisms that would allow public clients to be secured to the same degree and hence their use is not within the scope of this document.

Although it is possible to code authorization servers and clients from first principles using this document, implementers are encouraged to build on top of existing OpenID Connect and/or OAuth 2.0 implementations instead of embarking on a 'from scratch' implementation. See Section 6.6 for additional considerations for ensuring that implementations are complete and correct.

5.1.2. Profiling this document

This document is a general purpose high security profile of OAuth 2.0 that has been proved by formal analysis to meet the stated attacker model.

This document, and the underlying specifications, leave a number of choices open to implementors, deployers and/or ecosystems. With knowledge of the exact use cases, further reducing the number of choices may further improve security, or make implementation or interoperability easier.

However, for a profile to be compliant with this document, the profile shall not remove or override mandatory behaviors, as doing so is likely to invalidate the formal security analysis and reduce security in potentially unpredictable ways.

5.2. Network layer protections

5.2.1. Requirements for all endpoints

To protect against network attacks, clients, authorization servers, and resource servers

1. shall only offer TLS protected endpoints and shall establish connections to other servers using TLS;
2. shall set up TLS connections using TLS version 1.2 or later;
3. shall follow the recommendations for Secure Use of Transport Layer Security in [BCP195];
4. should use DNSSEC to protect against DNS spoofing attacks that can lead to the issuance of rogue domain-validated TLS certificates; and
5. shall perform a TLS server certificate check, as per [RFC9525].

NOTE 1: Even if an endpoint uses only organization validated (OV) or extended validation (EV) TLS certificates, an attacker using rogue domain-validated certificates is able to impersonate the endpoint and conduct man-in-the-middle attacks. CAA records [RFC8659] help to mitigate this risk.

5.2.2. Requirements for endpoints not used by web browsers

For server-to-server communication endpoints that are not used by web browsers, the following requirements apply:

1. When using TLS 1.2, servers shall only permit the cipher suites recommended in [BCP195];
2. When using TLS 1.2, clients should only permit the cipher suites recommended in [BCP195].

5.2.2.1. MTLS ecosystems

Some ecosystems may implement MTLS as an additional security control at the transport layer for all server-to-server endpoints requiring sensitive data being transmitted. For example, `private_key_jwt` can be used for client authentication in conjunction with MTLS connectivity. To facilitate interoperability:

- MTLS ecosystems should provide the trust list of the certificate authorities;
- authorization server implementations may utilize `mtls_endpoint_aliases` authorization server metadata as described in Section 5 of [RFC8705] to provide a discovery mechanism for endpoints that might have both MTLS and non-MTLS endpoints;
- client implementations shall use client metadata `use_mtls_endpoint_aliases` (see Section 5.2.2.1.1), if present, for endpoint communications.

5.2.2.1.1. Client Metadata

The Dynamic Client Registration Protocol [RFC7591] defines an API for dynamically registering OAuth 2.0 client metadata with authorization servers. The metadata defined by [RFC7591], and registered extensions to it, also imply a general data model for clients that is useful for authorization server implementations even when the dynamic client registration protocol isn't in play. Such implementations will typically have some sort of user interface available for managing client configuration.

The following client metadata parameter is introduced by this specification:

- `use_mtls_endpoint_aliases`:
  - OPTIONAL. Boolean value indicating the requirement for a client to use mutual-TLS endpoint aliases [RFC8705] declared by the authorization server in its metadata even beyond the Mutual-TLS Client Authentication and Certificate-Bound Access Tokens use cases. If omitted, the default value is false.

5.2.3. Requirements for endpoints used by web browsers

For endpoints that are used by web browsers, the following additional requirements apply:

1. Servers shall use methods to ensure that connections cannot be downgraded using TLS stripping attacks. A preloaded [preload] HTTP Strict Transport Security policy [RFC6797] can be used for this purpose. Some top-level domains, like `.bank` and `.insurance`, have set such a policy and therefore protect all second-level domains below them.
2. When using TLS 1.2, servers shall only use cipher suites allowed in [BCP195].
3. Servers shall not support CORS [CORS.Protocol] for the authorization endpoint, as clients must perform an HTTP redirect rather than access this endpoint directly.

NOTE 1: When using TLS1.2 endpoints used by web browsers can use any cipher suite allowed in [BCP195], whereas endpoints not used by web browsers can only use cipher suites recommended by [BCP195].

NOTE 2: New versions of [BCP195] will be published by the IETF periodically. At a minimum, implementors are expected to become compliant with newly issued versions of BCP195 within 12 months, or sooner.

5.3. Profile

5.3.1. General

In the following, a profile of the following technologies is defined:

- OAuth 2.0 Authorization Framework [RFC6749]
- OAuth 2.0 Bearer Tokens [RFC6750]
- Proof Key for Code Exchange by OAuth Public Clients (PKCE) [RFC7636]
- OAuth 2.0 Mutual-TLS Client Authentication and Certificate-Bound Access Tokens (MTLS) [RFC8705]
- OAuth 2.0 Demonstrating Proof of Possession (DPoP) [RFC9449]
- OAuth 2.0 Pushed Authorization Requests (PAR) [RFC9126]
- OAuth 2.0 Authorization Server Metadata [RFC8414]
- OAuth 2.0 Authorization Server Issuer Identification [RFC9207]
- OpenID Connect Core 1.0 incorporating errata set 1 [OIDC]

5.3.2. Requirements for authorization servers

5.3.2.1. General requirements

Authorization servers

1. shall distribute discovery metadata (such as the authorization endpoint) via the metadata document as specified in [OIDD] and [RFC8414];
2. shall reject requests using the resource owner password credentials grant;
3. shall only support confidential clients as defined in [RFC6749];
4. shall only issue sender-constrained access tokens;
5. shall use one of the following methods for sender-constrained access tokens:
   - MTLS as described in [RFC8705],
   - DPoP as described in [RFC9449];
6. shall authenticate clients using one of the following methods:
   - MTLS as specified in Section 2 of [RFC8705], or
   - `private_key_jwt` as specified in Section 9 of [OIDC];
7. shall not expose open redirectors (see Section 4.11 of [I-D.ietf-oauth-security-topics]);
8. shall only accept its issuer identifier value (as defined in [RFC8414]) as a string in the `aud` claim received in client authentication assertions;
9. shall not use refresh token rotation except in extraordinary circumstances (see Note 1 below);
10. if using DPoP, may use the server provided nonce mechanism (as defined in Section 8 of [RFC9449]);
11. shall issue authorization codes with a maximum lifetime of 60 seconds;
12. if using DPoP, shall support "Authorization Code Binding to DPoP Key" (as required by Section 10.1 of [RFC9449]);
13. to accommodate clock offsets, shall accept JWTs with an `iat` or `nbf` timestamp between 0 and 10 seconds in the future but shall reject JWTs with an `iat` or `nbf` timestamp greater than 60 seconds in the future. See Note 3 for further details and rationale; and
14. should restrict the privileges associated with an access token to the minimum required for the particular application or use case.

NOTE 1: The use of refresh token rotation does not provide security benefits when used with confidential clients and sender-constrained access tokens. This specification prohibits the use of refresh token rotation for security reasons as it causes user experience degradation and operational issues whenever the client fails to store or receive the new refresh token and has no option to retry.

However, as refresh token rotation may be required from time to time for infrastructure migration or similar extraordinary circumstances, this specification allows it, provided that authorization servers offer clients the time-limited option to retry with the old refresh token in case of failure. Implementers need to consider a secure mechanism for clients to recover from a loss of a new refresh token on issue. The details of this mechanism are outside the scope of this specification.

NOTE 2: This document is structured to support a variety of grants to be used with the general requirements above. For example the client credentials grant or the CIBA grant [CIBA]. Implementers should note that as of the time of writing only the authorization code flow and CIBA flows have been through a detailed security analysis [FAPI2SEC].

NOTE 3: Clock skew is a cause of many interoperability issues. Even a few hundred milliseconds of clock skew can cause JWTs to be rejected for being "issued in the future". The DPoP specification [RFC9449] suggests that JWTs are accepted in the reasonably near future (on the order of seconds or minutes). This document goes further by requiring authorization servers to accept JWTs that have timestamps up to 10 seconds in the future. 10 seconds was chosen as a value that does not affect security while greatly increasing interoperability. Implementers are free to accept JWTs with a timestamp of up to 60 seconds in the future. Some ecosystems have found that the value of 30 seconds is needed to fully eliminate clock skew issues. To prevent implementations switching off `iat` and `nbf` checks completely this document imposes a maximum timestamp in the future of 60 seconds.

5.3.2.2. Authorization endpoint flows

For flows that use the authorization endpoint, authorization servers

1. shall require the value of `response_type` described in [RFC6749] to be `code`;
2. shall support client-authenticated pushed authorization requests according to [RFC9126];
3. shall reject authorization requests sent without [RFC9126];
4. shall reject pushed authorization requests without client authentication;
5. shall require PKCE [RFC7636] with S256 as the code challenge method;
6. shall require the `redirect_uri` parameter in pushed authorization requests;
7. shall return an `iss` parameter in the authorization response according to [RFC9207];
8. shall not transmit authorization responses over unencrypted network connections, and, to this end, shall not allow redirect URIs that use the "http" scheme except for native clients that use loopback interface Redirection as described in Section 7.3 of [RFC8252];
9. shall reject an authorization code (Section 1.3.1 of [RFC6749]) if it has been previously used;
10. shall not use the HTTP 307 status code when redirecting a request that contains user credentials to avoid forwarding the credentials to a third party accidentally (see Section 4.12 of [I-D.ietf-oauth-security-topics]);
11. should use the HTTP 303 status code when redirecting the user agent using status codes;
12. shall issue pushed authorization requests `request_uri` with `expires_in` values of less than 600 seconds;
13. should provide end-users with all necessary information to make an informed decision about whether to consent to the authorization request, including the identity of the client and the scope of the authorization; and
14. if supporting [OIDC], shall support nonce parameter values up to 64 characters in length, may reject nonce values longer than 64 characters.

NOTE 1: If replay identification of the authorization code is not possible, it is desirable to set the validity period of the authorization code to one minute or a suitable short period of time. The validity period may act as a cache control indicator of when to clear the authorization code cache if one is used.

NOTE 2: The `request_uri` `expires_in` time must be sufficient for the user's device to receive the link and the user to complete the process of opening the link. In many cases (poor network connection or where the user has to manually select the browser to be used) this can easily take over 30 seconds.

NOTE 3: It is recommended that authorization servers that enforce one-time use of `request_uri` values ensure the enforcement takes place at the point of authorization, not at the point of loading an authorization page. This prevents user software that preloads urls from invalidating the `request_uri`.

NOTE 4: In this document the state parameter is not used for CSRF protection, but may be used to by the client for application state. In circumstances where clients encode application state in a JWT the length of the state parameter value could be in excess of 1000 characters.

NOTE 5: The use of OAuth 2.0 Rich Authorization Requests (RAR) [RFC9396] is recommended when the `scope` parameter is not expressive enough to convey the authorization that a client may want to obtain.

5.3.2.3. Returning authenticated user's identifier

If it is desired to provide the authenticated user's identifier to the client in the token response, the authorization server shall support OpenID Connect [OIDC].

5.3.3. Requirements for clients

5.3.3.1. General requirements

Clients

1. shall support sender-constrained access tokens using one or both of the following methods:
   - MTLS as described in [RFC8705],
   - DPoP as described in [RFC9449];
2. shall support client authentication using one or both of the following methods:
   - MTLS as specified in Section 2 of [RFC8705],
   - `private_key_jwt` as specified in Section 9 of [OIDC];
3. shall send access tokens in the HTTP header as in Section 2.1 of OAuth 2.0 Bearer Token Usage [RFC6750] or Section 7.1 of DPoP [RFC9449];
4. shall not expose open redirectors (see Section 4.11 of [I-D.ietf-oauth-security-topics]);
5. if using `private_key_jwt`, shall use the authorization server's issuer identifier value (as defined in [RFC8414]) in the `aud` claim in client authentication assertions. The issuer identifier value shall be sent as a string not as an item in an array.
6. shall support refresh tokens and their rotation;
7. if using MTLS client authentication or MTLS sender-constrained access tokens, shall support the `mtls_endpoint_aliases` metadata defined in [RFC8705];
8. if using DPoP, shall support the server provided nonce mechanism (as defined in Section 8 of [RFC9449]);
9. shall only use authorization server metadata (such as the authorization endpoint) retrieved from the metadata document as specified in [OIDD] and [RFC8414];
10. shall ensure that the issuer URL used as the basis for retrieving the authorization server metadata is obtained from an authoritative source and using a secure channel, such that it cannot be modified by an attacker;
11. shall ensure that this issuer URL and the `issuer` value in the obtained metadata match;
12. shall initiate an authorization process only with the end-user's explicit or implicit consent and protect initiation of an authorization process against cross-site request forgery, thereby enabling the end-user to

be aware of the context in which a flow was started; and
13. should request authorization with the least privileges necessary for the specific application or use case.

NOTE 1: This profile may be used by confidential clients on a user-controlled device where the system clock may not be accurate, causing `private_key_jwt` client authentication to fail. In such circumstances a client should consider using the HTTP date header returned from the server to synchronize its own clock when generating client assertions.

NOTE 2: Although authorization servers are required to support "Authorization Code Binding to DPoP Key" (as defined by Section 10.1 of [RFC9449]), clients are not required to use it.

5.3.3.2. Authorization code flow

For the authorization code flow, clients

1. shall use the authorization code grant described in [RFC6749];
2. shall use pushed authorization requests according to [RFC9126];
3. shall use PKCE [RFC7636] with `S256` as the code challenge method;
4. shall generate the PKCE challenge specifically for each authorization request and securely bind the challenge to the client and the user agent in which the flow was started;
5. shall check the `iss` parameter in the authorization response according to [RFC9207] to prevent mix-up attacks;
6. shall only send `client_id` and `request_uri` request parameters to the authorization endpoint (all other authorization request parameters are sent in the pushed authorization request according to [RFC9126]);
7. if using [OIDC], should not use nonce parameter values longer than 64 characters.

NOTE 1: The recommended restrictions on the nonce parameter value length is to aid interoperability.

5.3.4. Requirements for resource servers

The FAPI 2.0 endpoints are OAuth 2.0 protected resource endpoints that return protected information for the resource owner associated with the submitted access token.

Resource servers with the FAPI endpoints

1. shall accept access tokens in the HTTP header as in Section 2.1 of OAuth 2.0 Bearer Token Usage [RFC6750] or Section 7.1 of DPoP [RFC9449];
2. shall not accept access tokens in the query parameters stated in Section 2.3 of OAuth 2.0 Bearer Token Usage [RFC6750];
3. shall verify the validity, integrity, expiration and revocation status of access tokens;
4. shall verify that the authorization represented by the access token is sufficient for the requested resource access and otherwise return errors as in Section 3.1 of [RFC6750]; and
5. shall support and verify sender-constrained access tokens using one or both of the following methods:
   - MTLS as described in [RFC8705],
   - DPoP as described in [RFC9449].

5.4. Cryptography and secrets

5.4.1. General requirements

The following requirements apply to cryptographic operations and secrets:

1. Authorization servers, clients, and resource servers when creating or processing JWTs shall

    1. adhere to [RFC8725];
    2. use PS256, ES256, or EdDSA (using the Ed25519 variant) algorithms; and
    3. not use or accept the none algorithm.
2. RSA keys shall have a minimum length of 2048 bits.
3. Elliptic curve keys shall have a minimum length of 224 bits.
4. Credentials not intended for handling by end-users (e.g., access tokens, refresh tokens, authorization codes, etc.) shall be created with at least 128 bits of entropy such that an attacker correctly guessing the value is computationally infeasible. Cf. Section 10.10 of [RFC6749].

Note: As of the time of writing there isn't a registered fully-specified algorithm describing "EdDSA using the Ed25519 variant". If such algorithm is registered in the future, it is also allowed to be used for this profile.

5.4.2. JSON Web Key Sets

This profile supports the use of `private_key_jwt` and in addition allows the use of OpenID Connect. When these are used clients and authorization servers need to verify payloads with keys from another party. For authorization servers this profile strongly recommends the use of JWKS URI endpoints to distribute public keys. For client's key management this profile recommends either the use of JWKS URI endpoints or the use of the `jwks` parameter in combination with [RFC7591] and [RFC7592].

The definition of the authorization server `jwks_uri` can be found in [RFC8414], while the definition of the client `jwks_uri` can be found in [RFC7591].

In addition, any server providing a `jwks_uri` endpoint

1. shall only serve the `jwks_uri` endpoint over TLS;
2. should not use the JOSE headers for `x5u` and `jku`; and
3. should not serve a JWK set with multiple keys with the same `kid`.

5.4.3. Handling Duplicate Key Identifiers

JWK sets should not contain multiple keys with the same `kid`. However, to increase interoperability when there are multiple keys with the same `kid`, the verifier shall consider other JWK attributes, such as `kty`, `use`, `alg`, etc., when selecting the verification key for the particular JWS message. For example, the following algorithm could be used in selecting which key to use to verify a message signature:

1. find keys with a `kid` that matches the `kid` in the JOSE header;
2. if a single key is found, use that key;
3. if multiple keys are found, then the verifier should iterate through the keys until a key is found that has a matching `alg`, `use`, `kty`, or `crv` that corresponds to the message being verified.

5.5. Main differences to FAPI 1.0

FAPI 1.0 - Part 2: Advanced	FAPI 2.0	Reasons
JAR	PAR	integrity protection and compatibility improvements for authorization requests
JARM	only code in response	the authorization response is reduced to only contain the authorization code, obsoleting the need for integrity protection

FAPI 1.0 - Part 2: Advanced	FAPI 2.0	Reasons
BCM principles, defences based on particular threats	attacker model, security goals, best practices from the OAuth Security BCP	clearer design guideline, suitability for formal analysis
`s_hash`	PKCE	protection provided by `state` (in particular against CSRF) is now provided by PKCE; `state` integrity is partially protected by PAR
pre-registered redirect URIs	redirect URIs in PAR	pre-registration is not required with client authentication and PAR
response types `code` `id_token` or `code`	response type `code`	no ID token in front-channel (privacy improvement); nonce/signature check can be skipped by clients, PKCE cannot (security improvement)
ID Token as detached signature	PKCE	ID token does not need to serve as a detached signature
potentially encrypted ID Tokens in the front channel	No ID Tokens in the front channel (therefore no encryption required)	ID Tokens are only exchanged in the back channel and as such do not need to be encrypted
`nbf` & `exp` claims in request object	`request_uri` has limited lifetime	Prevents pre-generation of requests
`x-fapi-*` headers	Moved to Implementation and Deployment Advice document	Not relevant to the core of the security profile
MTLS for sender-constrained access tokens	MTLS or DPoP	Due to the lack of the tight integration with the TLS layer, DPoP can be easier to deploy in some scenarios

Table 1

6. Security considerations

6.1. Access token lifetimes

The use of short-lived access tokens (combined with refresh tokens) potentially reduces the time window for some attacks.

The use of refresh tokens also allows clients to rotate their sender-constraining keys without loss of grants, either because of compromise of the key or as part of good security hygiene.

If issuing long-lived grants (e.g. days/weeks), consider using short-lived (e.g. minutes/hours) access tokens combined with refresh tokens.

There is a performance and resiliency trade-off, setting the access token lifetime too short can increase the load on and dependency on the authorization server.

6.2. DPoP proof replay

An attacker of type A5 (see [attackermodel]) may be able to obtain DPoP proofs that they can then replay.

This may also allow reuse of the DPoP proof with an altered request, as DPoP does not sign the body of HTTP requests nor most headers. For example, for a payment request the attacker might be able to specify a different amount or destination account.

Possible mitigations for this are:

1. Resource servers use short-lived DPoP nonces to reduce the time window where a request can be replayed.
2. Resource servers implement replay prevention using the `jti` header as explained in [RFC9449].
3. Replay of an altered request can be prevented by using signed resource requests as per FAPI Message Signing [FAPIMessageSigning].
4. Consider MTLS sender-constraining instead of DPoP.

These mitigations may have potential complexity, performance or scalability trade-offs. Attacker type A5 represents a powerful attacker and mitigations may not be necessary for many ecosystems.

6.3. Injection of stolen access tokens

There are potential situations where the attacker may be able to inject stolen access tokens into a client to bypass [RFC8705] or [RFC9449] sender-constraining of the access token, as described in "Cuckoo's Token Attack" in [FAPI1SEC].

A pre-condition for this attack is that the attacker has control of an authorization server that is trusted by the client to issue access tokens for the target resource server. An attacker may obtain control of an authorization server by:

1. compromising the security of a different authorization server that the client trusts;
2. acting as an authorization server and establishing a trust relationship with a client using social engineering; or
3. compromising the client.

The attack may be easier if a centralized directory or other resource server discovery mechanism allows the attacker to cause the client to send the stolen access token received from the attacker-controlled authorization server to an honest resource server.

The pre-conditions for this attack do not apply to many ecosystems and require a powerful attacker. In situations where the pre-conditions may be met, the possible mitigations include:

1. clients using different DPoP keys or MTLS certificates at each authorization server;
2. clients sending the issuer identifier the access token was obtained from to the resource server, and requiring resource servers to verify the issuer matches the authorization server that originally issued the token (though there is no standardized method for clients to send the issuer to the resource server);
3. reducing the time window for the attack by using short-lived access tokens alongside refresh tokens.

6.4. Authorization request leaks lead to CSRF

An attacker of type A3 (see [attackermodel]) can intercept an authorization request, log in at the authorization server, receive an authorization code and redirect the honest user via a cross-site request forgery (CSRF) attack to the honest client but with the attacker's authorization code. This results in the user accessing the attacker's resources, thus breaking session integrity.

It is important to note that all practically used redirect-based flows are susceptible to this attack, as redirection does not allow for a tight coupling of the session between the user's browser and the client on the one side and the session between the user's browser and the authorization server on the other side. This attack, however, requires a strong attacker who can read authorization requests and perform a CSRF attack in a short time window.

Possible mitigations for this are:

1. Requiring the authorization server to only accept a `request_uri` once. This will prevent attacks where the attacker was able to read the authorization request, but not use the `request_uri` before the honest user does so.
2. Requiring the client to only make one authorization code grant call for each authorization endpoint call. This will prevent attacks where the attacker was unable to send the authorization response before the honest user does so.
3. Reducing the lifetime of the authorization code - this will reduce the window in which the CSRF attack has to be performed.

An attacker that has the option to block a user's request completely can circumvent the first and second defences. In practice, however, attackers can often read an authorization request (e.g., from a log file or via some other side-channel), but not block the request from being sent. If the victim's internet connection is slow, this might increase the attacker's chances.

6.5. Browser-swapping attacks

An attacker that has access to the authorization response sent through a victim's browser can perform a browser-swapping attack as follows:

1. The attacker starts a new flow using their own browser and some client. The client sends a pushed authorization request to the authorization server and receives a `request_uri` in the response. The client then redirects the attacker's browser to the authorization server.
2. The attacker intercepts this redirection and forwards the URL to a victim. For example, the attacker can embed a link to this URL in a phishing website, an email, or a QR code.
3. The victim may be tricked into believing that an authentication/authorization is legitimately required. The victim therefore authenticates at the authorization server and may grant the client access to their data.
4. The attacker can now intercept the authorization response in the victim's browser and forward it to the client using their own browser.
5. The client will recognize that the authorization response belongs to the same browser that initially started the transaction (the attacker's browser) and exchange the authorization code for an access token and/or obtain user information.
6. Via the client, the attacker now has access to the user's resources or is logged in as the user.

With currently deployed technology, there is no way to completely prevent this attack if the authorization response leaks to an attacker in any redirect-based protocol. It is therefore important to keep the authorization response confidential. The requirements in this security profile are designed to achieve that, e.g., by disallowing open redirectors and requiring that the `redirect_uri` is sent via an authenticated and encrypted channel, the pushed authorization request, ensuring that the `redirect_uri` cannot be manipulated by the attacker.

Implementers need to consider the confidentiality of the authorization response critical when designing their systems, in particular when this security profile is used in other contexts, e.g., mobile applications.

6.6. Incomplete or incorrect implementations of the specifications

To achieve the full security and interoperability benefits, it is important that the implementation of this document and the underlying OpenID Connect and OAuth specifications is both complete and correct.

The OpenID Foundation provides tools that can be used to confirm that an implementation is correct:

https://openid.net/certification/

The OpenID Foundation maintains a list of certified implementations:

https://openid.net/developers/certified/

Deployments that use this document should use certified implementations.

6.7. Client Impersonating Resource Owner

Section 4.15 of [I-D.ietf-oauth-security-topics] describes an attack where a malicious client is able to influence its client_id such that it could be mistaken for an end-user subject identifier. This attack also requires that an authorization server issues access tokens with similar privileges to both clients and end-users.

For this reason, authorization servers should not allow clients to influence their client_id in a way that it can be mistaken for an end-user subject identifier.

6.8. Key Compromise

In the event that a cryptographic key is compromised, it is important to limit the impact of such an event. This can be achieved by:

1. Key rotation: automated regular key rotation is recommended, as it reduces the time window in which a compromised key can be used. jwks_uri endpoints allows parties to rotate their keys without the need for manual, error-prone coordination.
2. Key scope: single purpose keys are recommended. For example, it is not recomended to use the same key for signing and encryption. See Section 5.2 of [NIST.SP.800-57pt1r5] for further guidance.
3. Stateful credentials: It is recommended that implementers consider the trade-offs between stateful and stateless credentials, such as access tokens. In the event of a key compromise, the use of stateless tokens signed by the compromised key could enable an attacker to forge tokens. This risk can be mitigated if all tokens are stateful, meaning there is a mechanism to validate each token's active status through a central authority or database. However, stateless tokens offer significant advantages. They carry all necessary information within themselves, improving performance by removing the need for server-side database lookups and eliminating central session data storage. Additionally, they can be parsed and validated by

resource servers directly, without further authorization server involvement. This enhances scalability and flexibility, particularly in scenarios where the authorization server and resource server are not co-located or managed by the same entity (as discussed in the introduction to [RFC9068]).

4. Credential linking: When multiple credentials are issued as part of the same authorization, it is recommended that their relationship be explicitly established and recorded. This way, if one credential in a linked set is compromised, all related credentials can be revoked.

7. Privacy considerations

There are many factors to be considered in terms of privacy when implementing this document. Since this document is a profile of OAuth 2.0 and OpenID Connect, the privacy considerations are not specific to this document and generally apply to OAuth or OpenID Connect. Implementers are advised to perform a thorough privacy impact assessment and manage identified risks appropriately.

NOTE 1: Implementers can consult documents like [ISO29100] and [ISO29134] for this purpose.

Privacy threats to OAuth and OpenID Connect implementations include the following:

- Inappropriate privacy notice: A privacy notice (e.g., provided at a `policy_url`) or by other means can be inappropriate or insufficient.
- Inadequate choice: Providing a consent screen without adequate choices does not form consent.
- Misuse of data: An authorization server, resource server or client can potentially use the data not according to the purpose that was agreed.
- Collection minimization violation: A client asking for more data than it absolutely needs to fulfill the purpose is violating the collection minimization principle.
- Unsolicited personal data from the resource server: Some bad resource server implementations may return more data than requested. If the data is personal data, then this would be a violation of privacy principles.
- Data minimization violation: Any process that is processing more data than it needs is violating the data minimization principle.
- Authorization servers tracking end-users: Authorization servers identifying what data is being provided to which client for which end-user.
- End-user tracking by clients: Two or more clients correlating access tokens or ID Tokens to track users.
- Client misidentification by end-users: End-user misunderstands who the client is due to a confusing representation of the client at the authorization server's authorization page.
- Insufficient understanding of the end-user granting access to data: To enhance the trust of the ecosystem, best practice is for the authorization server to make clear what is included in the authorization request (for example, what data will be released to the client).
- Attacker observing personal data in authorization request/response: The authorization request or response might contain personal data. In some jurisdictions, even security parameters can be considered personal data. This profile aims to reduce the data sent in the authorization request and response to an absolute minimum, but nonetheless, an attacker might observe some data.
- Data leak from authorization server: The authorization server generally stores personal data. If it becomes compromised, this data can leak or be modified.
- Data leak from resource servers: Some resource servers store personal data. If a resource server becomes compromised, this data can leak or be modified.
- Data leak from clients: Some clients store personal data. If the client becomes compromised, this data can leak or be modified.

8. IANA Considerations

8.1. OAuth Dynamic Client Registration Metadata registration

This specification requests registration of the following client metadata definitions in the IANA "OAuth Dynamic Client Registration Metadata" registry established by [RFC7591]:

8.1.1. Registry Contents

- Client Metadata Name: `use_mtls_endpoint_aliases`
- Client Metadata Description: Boolean value indicating the requirement for a client to use mutual-TLS endpoint aliases [RFC8705] declared by the authorization server in its metadata even beyond the Mutual-TLS Client Authentication and Certificate-Bound Access Tokens use cases.
- Change Controller: OpenID Foundation FAPI Working Group - openid-specs-fapi@lists.openid.net
- Specification Document(s): Section 5.2.2.1.1 of this specification

9. Normative References

[BCP195] IETF, "BCP195", <https://www.rfc-editor.org/info/bcp195>.

[CORS.Protocol] WHATWG, "CORS Protocol", <https://fetch.spec.whatwg.org/#http-cors-protocol>.

[ISO29100] ISO/IEC, "ISO/IEC 29100 Information technology – Security techniques – Privacy framework", <https://www.iso.org/standard/85938.html>.

[OIDC] Sakimura, N., Bradley, J., Jones, M., de Medeiros, B., and C. Mortimore, "OpenID Connect Core 1.0 incorporating errata set 1", 8 November 2014, <http://openid.net/specs/openid-connect-core-1_0.html>.

[OIDD] Sakimura, N., Bradley, J., Jones, M., and E. Jay, "OpenID Connect Discovery 1.0 incorporating errata set 1", 8 November 2014, <https://openid.net/specs/openid-connect-discovery-1_0.html>.

[RFC6749] Hardt, D., Ed., "The OAuth 2.0 Authorization Framework", RFC 6749, DOI 10.17487/RFC6749, October 2012, <https://www.rfc-editor.org/info/rfc6749>.

[RFC6750] Jones, M. and D. Hardt, "The OAuth 2.0 Authorization Framework: Bearer Token Usage", RFC 6750, DOI 10.17487/RFC6750, October 2012, <https://www.rfc-editor.org/info/rfc6750>.

[RFC7636] Sakimura, N., Ed., Bradley, J., and N. Agarwal, "Proof Key for Code Exchange by OAuth Public Clients", RFC 7636, DOI 10.17487/RFC7636, September 2015, <https://www.rfc-editor.org/info/rfc7636>.

[RFC8252] Denniss, W. and J. Bradley, "OAuth 2.0 for Native Apps", BCP 212, RFC 8252, DOI 10.17487/RFC8252, October 2017, <https://www.rfc-editor.org/info/rfc8252>.

[RFC8414] Jones, M., Sakimura, N., and J. Bradley, "OAuth 2.0 Authorization Server Metadata", RFC 8414, DOI 10.17487/RFC8414, June 2018, <https://www.rfc-editor.org/info/rfc8414>.

[RFC8725] Sheffer, Y., Hardt, D., and M. Jones, "JSON Web Token Best Current Practices", BCP 225, RFC 8725, DOI 10.17487/RFC8725, February 2020, <https://www.rfc-editor.org/info/rfc8725>.

[RFC9126] Lodderstedt, T., Campbell, B., Sakimura, N., Tonge, D., and F. Skokan, "OAuth 2.0 Pushed Authorization Requests", RFC 9126, DOI 10.17487/RFC9126, September 2021, <https://www.rfc-editor.org/info/rfc9126>.

[RFC9207] Meyer zu Selhausen, K. and D. Fett, "OAuth 2.0 Authorization Server Issuer Identification", RFC 9207, DOI 10.17487/RFC9207, March 2022, <https://www.rfc-editor.org/info/rfc9207>.

[RFC9449] Fett, D., Campbell, B., Bradley, J., Lodderstedt, T., Jones, M., and D. Waite, "OAuth 2.0 Demonstrating Proof of Possession (DPoP)", RFC 9449, DOI 10.17487/RFC9449, September 2023, <https://www.rfc-editor.org/info/rfc9449>.

[RFC9525] Saint-Andre, P. and R. Salz, "Service Identity in TLS", RFC 9525, DOI 10.17487/RFC9525, November 2023, <https://www.rfc-editor.org/info/rfc9525>.

[RFC9700] Lodderstedt, T., Bradley, J., Labunets, A., and D. Fett, "Best Current Practice for OAuth 2.0 Security", BCP 240, RFC 9700, DOI 10.17487/RFC9700, January 2025, <https://www.rfc-editor.org/info/rfc9700>.

[attackermodel] Fett, D., "FAPI 2.0 Attacker Model", 18 February 2025, <https://openid.net/specs/fapi-attacker-model-2_0-final.html>.

10. Informative References

[CIBA] Fernandez Rodriguez, G., Walter, F., Nennker, A., Tonge, D., and B. Campbell, "OpenID Connect Client-Initiated Backchannel Authentication Flow - Core 1.0", 1 September 2021, <http://openid.net/specs/openid-client-initiated-backchannel-authentication-core-1_0.html>.

[FAPI1SEC] Fett, D., Hosseyni, P., and R. Kuesters, "An Extensive Formal Security Analysis of the OpenID Financial-grade API", 31 January 2019, <https://arxiv.org/abs/1901.11520>.

[FAPI2SEC] Hosseyni, P., Kuesters, R., and T. Würtele, "Formal Security Analysis of the OpenID Financial-grade API 2.0", 8 July 2024, <https://doi.ieeecomputersociety.org/10.1109/CSF61375.2024.00002>.

[FAPIMessageSigning] Tonge, D. and D. Fett, "FAPI 2.0 Message Signing", 17 January 2024, <https://openid.net/specs/fapi-2_0-message-signing-ID1.html>.

[I-D.ietf-oauth-security-topics] Lodderstedt, T., Bradley, J., Labunets, A., and D. Fett, "OAuth 2.0 Security Best Current Practice", Work in Progress, Internet-Draft, draft-ietf-oauth-security-topics-29, 3 June 2024, <https://datatracker.ietf.org/doc/html/draft-ietf-oauth-security-topics-29>.

[ISO29134] ISO/IEC, "ISO/IEC 29134 Information technology – Security techniques – Guidelines for privacy impact assessment", <https://www.iso.org/standard/86012.html>.

[ISODIR2] ISO/IEC, "ISO/IEC Directives, Part 2 - Principles and rules for the structure and drafting of ISO and IEC documents", <https://www.iso.org/sites/directives/current/part2/index.xhtml>.

[NIST.SP.800-57pt1r5] Barker, E. and A. Roginsky, "NIST Special Publication 800-57 Part 1 Revision 5", 1 May 2020, <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-57pt1r5.pdf>.

[RFC6797] Hodges, J., Jackson, C., and A. Barth, "HTTP Strict Transport Security (HSTS)", RFC 6797, DOI 10.17487/RFC6797, November 2012, <https://www.rfc-editor.org/info/rfc6797>.

[RFC7591] IETF, "OAuth 2.0 Dynamic Client Registration Protocol", <https://datatracker.ietf.org/doc/html/rfc7591>.

[RFC7592] Richer, J., Ed., Jones, M., Bradley, J., and M. Machulak, "OAuth 2.0 Dynamic Client Registration Management Protocol", RFC 7592, DOI 10.17487/RFC7592, July 2015, <https://www.rfc-editor.org/info/rfc7592>.

[RFC8659] Hallam-Baker, P., Stradling, R., and J. Hoffman-Andrews, "DNS Certification Authority Authorization (CAA) Resource Record", RFC 8659, DOI 10.17487/RFC8659, November 2019, <https://www.rfc-editor.org/info/rfc8659>.

[RFC8705] Campbell, B., Bradley, J., Sakimura, N., and T. Lodderstedt, "OAuth 2.0 Mutual-TLS Client Authentication and Certificate-Bound Access Tokens", RFC 8705, DOI 10.17487/RFC8705, February 2020, <https://www.rfc-editor.org/info/rfc8705>.

[RFC9068] Bertocci, V., "JSON Web Token (JWT) Profile for OAuth 2.0 Access Tokens", RFC 9068, DOI 10.17487/RFC9068, October 2021, <https://www.rfc-editor.org/info/rfc9068>.

[RFC9396] Lodderstedt, T., Richer, J., and B. Campbell, "OAuth 2.0 Rich Authorization Requests", RFC 9396, DOI 10.17487/RFC9396, May 2023, <https://www.rfc-editor.org/info/rfc9396>.

[preload] Anonymous, "HSTS Preload List Submission", <https://hstspreload.org/>.

Appendix A. Acknowledgements

This document was developed by the OpenID FAPI Working Group.

We would like to thank Takahiko Kawasaki, Filip Skokan, Nat Sakimura, Stuart Low, Dima Postnikov, Torsten Lodderstedt, Travis Spencer, Brian Campbell, Ralph Bragg, Łukasz Jaromin, Pedram Hosseyni, Ralf Küsters, Tim Würtele, Edmund Jay, Aaron Parecki and Hideki Ikeda for their valuable feedback and contributions that helped to evolve this document.

Appendix B. Notices

Property Rights policy (found at openid.net) requires contributors to offer a patent promise not to assert certain patent claims against other contributors and against implementers. OpenID invites any interested party to bring to its attention any copyrights, patents, patent applications, or other proprietary rights that may cover technology that may be required to practice this specification.

Authors' Addresses

Daniel Fett
Authlete
Email: mail@danielfett.de

Dave Tonge
Moneyhub Financial Technology
Email: dave@tonge.org

Joseph Heenan
Authlete
Email: joseph@authlete.com